

Chapter 5 – Types of Functions

SLIDE 1 — What Are the Types of Functions?

Definition

A function type describes how a function receives data and whether it returns a value.

In C, functions are commonly classified into four types based on:

1. Whether the function has parameters.
2. Whether the function returns a value.

The Four Types

Type	Parameters	Return Value
Type 1	No	No
Type 2	Yes	No
Type 3	No	Yes
Type 4	Yes	Yes

Analogy

Think of a food shop:

- No input, no output: You press a button and receive nothing.
- Input, no output: You give an order and the shop prepares it.
- No input, output: You receive a fixed food item.
- Input and output: You give ingredients and receive a prepared dish.

Small Example

None

```
void greet() {  
    printf("Hello");  
}
```

This function has no parameter and no return value.

SLIDE 2 — Type 1: No Parameters and No Return Value

Meaning

This type of function:

- Does not receive any value.
- Does not return any value.
- Performs a fixed task.
- Usually uses `void` as the return type.

Why Is It Used?

It is useful when the function always performs the same action, such as:

- Displaying a message.
- Printing a menu.
- Showing a title.
- Displaying instructions.

Basic Pattern

None

```
void functionName() {  
    // statements  
}
```

Example

None

```
void greet() {  
    printf("Hello Student");  
}
```

Important Statement ★

This function performs a task without receiving input or returning a result.

SLIDE 3 — Type 2: Parameters and No Return Value

Meaning 🧠

This type of function:

- Receives one or more values.
- Uses parameters to process the values.
- Does not return a result.
- Commonly uses `void`.

Why Is It Used?

It is useful when:

- The function needs input.
- The result can be displayed directly.
- A task must be performed using different values.
- Repeated code needs to be reduced.

Basic Pattern

None

```
void functionName(int value) {  
    // statements  
}
```

Example 💻

None

```
void displaySquare(int n) {
```

```
printf("%d", n * n);  
}
```

Important Statement ★

The function receives data but does not send a value back.

SLIDE 4 — Type 3 and Type 4

Type 3: No Parameters but Return Value

Meaning

This function:

- Does not receive input.
- Performs a task.
- Returns a value to the caller.

Example

```
None  
int getNumber() {  
    return 10;  
}
```

Used For

- Returning fixed values.
- Providing default values.
- Getting information from a function.

Type 4: Parameters and Return Value

Meaning

This function:

- Receives one or more values.
- Processes the values.
- Returns the final result.

- Is commonly used in calculations.

Example

None

```
int add(int a, int b) {  
    return a + b;  
}
```

Used For

- Calculations.
- Reusable logic.
- Returning results.
- Building larger programs.

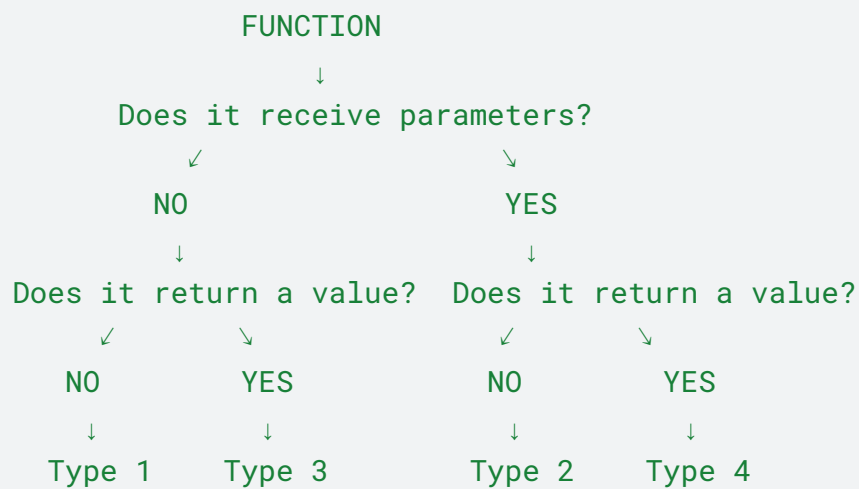
Important Statement ★

This is the most flexible type because it accepts input and returns output.

SLIDE 5 — Flowchart and Recap

Function Classification Flowchart

None



Quick Comparison ★

Type	Example Pattern
------	-----------------

Type 1	<code>void greet()</code>
--------	---------------------------

Type 2	<code>void display(int n)</code>
--------	--------------------------------------

Type 3	<code>int getNumber()</code>
--------	------------------------------

Type 4	<code>int add(int a, int b)</code>
--------	--

Witty Memory Trick 😄

No input, no output: Silent worker
Input, no output: Worker who completes the task
No input, output: Vending machine
Input and output: Smart calculator

Questions

1.How many common types of functions are there in C based on parameters and return values?

A. 2 B. 3 C. 4 D. 5

💡 Hint: Think about parameter and return value combinations.

📖 Glossary: Types — Different categories or forms.

Answer: C. 4

2.

None

```
#include <stdio.h>
```

```
void greet() {  
    printf("Hello");  
}  
  
int main() {  
    greet();  
    return 0;  
}
```

What is the output?

💡 Hint: The function is called inside `main()`.

📖 Glossary: Call — Asking a function to execute its code.

Answer:

None
`Hello`

3. Fill the Missing Code

```
None  
#include <stdio.h>  
  
void display() {  
    printf("C Programming");  
}  
  
int main() {  
    -----;  
    return 0;  
}
```

💡 Hint: Call the function using its name.

📖 Glossary: Function call — Using a function name to execute its code.

Answer:

None

`display()`

4. Jumbled Sentence

Arrange the words correctly:

value / a function / can / return / A

💡 Hint: Start with “A function”.

📖 Glossary: Return — To send a value back from a function.

Answer: A function can return a value.

5. Fill the Missing Code

None

```
#include <stdio.h>
```

```
void showMessage(_____ message) {  
    printf("%s", message);  
}
```

💡 Hint: The parameter stores text.

📖 Glossary: String — A group of characters used to store text.

Answer:

None

`char*`